

Shaping Musical Texture through Markov-Matrix Operations in Csound

Micael Antunes¹ *

Arts and Humanities Institute/ Music Department - Maynooth University
micael.antunesdasilva@mu.ie

Abstract. This paper presents four Csound opcodes for treating a Markov transition matrix as an explicit compositional object: **BuildBiasedMatrix** generates a matrix from intuitive affinity scores rather than typed probabilities, **MatRotate** cyclically shifts which state a hierarchy privileges, **MatInterp** interpolates between two matrices, and **MatEntropy** relaxes a matrix toward the maximum-entropy uniform matrix. These operations formalize procedures developed by hand across earlier textural compositions, now reusable across symbolic and audio-based work alike, regardless of the specific chain or material involved. Each operation is demonstrated through a short musical example built on a shared nine-state chain, with transition-matrix heatmaps, state timelines, and a Shannon-entropy curve offered as artifacts that consolidate the compositional intent behind each transformation. The result is a small, composition-first toolkit for shaping musical texture, situated within a practice-based approach to computer-assisted musical composition.

Keywords: computer-assisted composition; musical texture; Markov models

1 Introduction

This article presents a set of matrix operations for composing with Markov chains, implemented as opcodes in Csound [6]. Each operation transforms a row-stochastic transition matrix, reshaping how a chain moves a piece through its musical material over time. My compositional practice is anchored in musical texture and sound masses, within a computer-assisted approach [10]. Markov chains have long served this and related repertoires: Xenakis used them in *Analogique* to organize successions of sound states [13], and they have been studied as a general compositional model [1,11]. They remain widely used today, often within AI-assisted composition systems [8,4,5,12,9]. Changing a transition matrix over time, and nesting chains within chains, are long recognized in the Markov-composition literature [1]; the contribution here is a specific, Csound-native vocabulary of four such operations, derived from my own compositional practice.

I have used Markov chains manually in two earlier pieces. In *Rarefações* (2020), for snare drum and live electronics, transition matrices both shaped the density of the electronic sound masses and sequenced the pre-composed material of the score, shifting the texture from mass-based to rhythmic gestures across the piece [2]. In *Gestos* (2022), Markov chains organized the score's gestures, managing textural variety from section to section [10]. In both works I wrote the matrices manually, with some automation in Max/MSP using the bach library [3]. Working this way made a limitation clear: I needed these procedures as operations reusable across my own practice, whether the output is a symbolic score or audio. I built four, each answering a recurring compositional demand: 1) generating a matrix from intuitive affinities rather than typed probabilities (**BuildBiasedMatrix**); 2) shifting which hierarchy governs the chain without redesigning it (**MatRotate**); 3) transitioning gradually between two textures (**MatInterp**); and 4) moving a texture between determinate and indeterminate behavior (**MatEntropy**). The article begins by defining these operations formally; it then demonstrates each through a short musical example; finally, it situates them within a practice-based approach to composition.

* Funded by the European Union (MSCA Postdoctoral Fellowship GenCAST, No 101277782). Views and opinions expressed are however those of the author only and do not necessarily reflect those of the European Union or the granting authority. Neither the European Union nor the granting authority can be held responsible for them.

2 Matrix Operations

All operations below act on *row-stochastic* $N \times N$ matrices – transition matrices of a discrete-time Markov chain, $M \in \mathbb{R}^{N \times N}$ with $M_{i,j} \geq 0$ and $\sum_{j=0}^{N-1} M_{i,j} = 1$ for every row i . States are indexed $0, \dots, N-1$. They are implemented as Csound opcodes operating on flattened $N \times N$ function tables (`ftgen`), so no data structure beyond the standard Csound table is needed. Full opcode source is available in the repository.¹

Generation (BuildBiasedMatrix). Rather than hand-typing every transition probability, a matrix can be generated from two *affinity score* vectors via softmax: a target boost $\mathbf{b} \in \mathbb{R}^N$ (a general per-destination-state pull, applied to every row) and a self boost $\mathbf{s} \in \mathbb{R}^N$ (an extra tendency for state i to stay in itself, applied only on the diagonal), together with a temperature $T > 0$:

$$\text{score}_{i,j} = b_j + s_i \cdot [i=j], \quad M_{i,j} = \frac{\exp(\text{score}_{i,j}/T)}{\sum_{k=0}^{N-1} \exp(\text{score}_{i,k}/T)} \quad (1)$$

where $[i=j]$ is the Iverson bracket. Temperature controls sharpness: as $T \rightarrow 0^+$, each row concentrates on the destination(s) with maximal score; if the maximum is unique, the row approaches a one-hot distribution. As $T \rightarrow \infty$ every row approaches the uniform distribution, independent of the scores.

Rotation (MatRotate). To derive a second, structurally related matrix from an existing one without specifying new affinity scores, `MatRotate` cyclically permutes states by a shift k . In the *state relabeling* mode used in this paper, rows and columns are shifted together – equivalent to renaming every state $i \mapsto \sigma_k(i)$, with $\sigma_k(x) = (x + k) \bmod N$:

$$M_{\sigma_k(i), \sigma_k(j)} = A_{i,j} \quad (2)$$

which relocates A 's whole relational structure without altering it. The matrix B used in Sect. 3 is A rotated this way with $k = 5$.

Interpolation (MatInterp). Given two matrices A, B , `MatInterp` blends them at position $t \in [0, 1]$ ($t=0$ gives A , $t=1$ gives B):

$$M_{i,j} = (1 - t) A_{i,j} + t B_{i,j} \quad (3)$$

an affine combination of two row-stochastic matrices, so M is automatically row-stochastic with no renormalization needed.

Entropy Relaxation (MatEntropy). `MatEntropy` pushes a matrix A toward the maximum-entropy uniform matrix U ($U_{i,j} = 1/N$ for all i, j), by calling `MatInterp` (Eq. 3) with $B = U$:

$$\text{MatEntropy}(A, \alpha) = (1 - \alpha) A + \alpha U, \quad \alpha \in [0, 1] \quad (4)$$

$\alpha=0$ returns A unchanged; $\alpha=1$ returns U .

Usage. A matrix is generated from two affinity vectors and then performed:

```
giN init 9 ; 9 states
giTgt ftgen 0,0,-giN,-2, 1,1,2,6,2,1,2,5,2 ; per-state pull
giSlf ftgen 0,0,-giN,-2, 0,0,0,-2,0,0,0,-4,0 ; self-loop bias

instr Play
  iA      BuildBiasedMatrix giTgt, giSlf, 2.0 ; 2.0 = temperature
  itime, iend PlayChain iA, giN, 3, 80, 0, 0.5 ; start state 3, 80 visits
endin
```

Here `PlayChain` walks the chain, triggering the Csound instrument that renders each visited state. Generation and sound happen in the same environment, with no export step in between.

3 Musical Demonstrations

Each operation in Sect. 2 was built to answer a specific, recurring demand of the composer's own practice, not as a general-purpose Markov-chain toolkit exercise. The four musical examples below

¹ <https://github.com/MicaAnt/MatrixGen/tree/main/MatrixGen/orc>

pair each operation with the compositional demand it realizes, using a shared 9-state chain and material rendered through a sampled piano.² Each state is a short musical gesture, from a single note to a run of three or four, differing in register, contour, and rhythm so that the states are audibly distinct. What the chain traverses is therefore a sequence of gestures, not individual pitches; rendered audio for all four examples is available in the repository.³ We additionally track each matrix’s average row-wise Shannon entropy, $H(M)$, in bits, as a proxy for how spread out its transitions are, used throughout this section.⁴

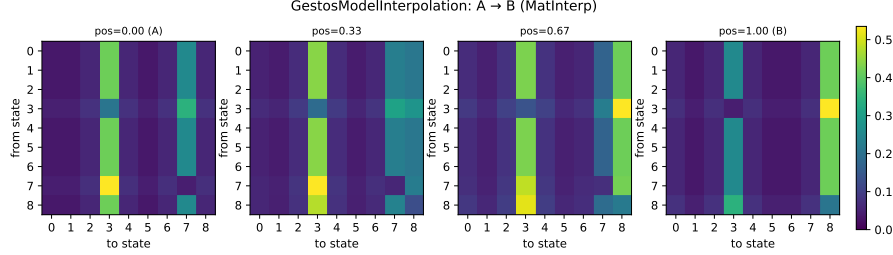


Fig. 1. Transition matrix at each of the 4 interpolation steps ($t = 0.00, 0.33, 0.67, 1.00$), from A (left) to B (right). Brighter (yellow) cells indicate higher transition probability; rows are origin states, columns are destination states.

Example 1 — Generation. A matrix can be generated directly from compositional intuition – which states should dominate, and by how much – without declaring every transition probability by hand. To exercise this workflow end-to-end, this example reuses the 9-state chain and per-state material from an earlier piece, *Gestos* (2022),⁵ but with entirely new weights: matrix A is generated via `BuildBiasedMatrix`, privileging state 3 (target boost 6, versus 1–2 elsewhere) with a negative self boost at states 3 and 7 (–2 and –4) discouraging self-loops there, at temperature $T=2.0$. The chain is performed by `PlayChain`.⁶ The leftmost panel of Fig. 1 and the *MatrizA* panel of Fig. 2 consolidate the result: state 3 does dominate in performance.

Example 2 — Rotation. A recurring need is to shift *which* hierarchy is in force partway through an example, without redesigning its shape from scratch. Matrix B is A rotated with $k = 5$, moving the privileged state from 3 to 8 while carrying the whole relational structure – including the self-loop suppression – along with it. The rightmost panel of Fig. 1 and the *MatrizB* panel of Fig. 2 show state 8 dominating in its place.

Example 3 — Interpolation. Textures should be able to move gradually from one hierarchy to another, rather than cut abruptly between them. This example interpolates $A \rightarrow B$ over 4 steps, with `PlayChain` re-invoked at each new matrix. The *Interpolation* panel of Fig. 2 shows the chain’s state carrying over continuously across each boundary (marked by the dotted lines) – the transformation is audible as a shift in tendency, not a hard cut. Fig. 4 adds that the entropy at both endpoints is identical (≈ 2.488 bits, since entropy is rotation-invariant), while the middle passes through a *more* spread out, less directed texture than either endpoint.

Example 4 — Entropy Relaxation. Determinacy versus indeterminacy is itself a compositional axis: a texture can be pushed toward a directed, predictable state or loosened toward an undirected one. This example relaxes from the uniform matrix back to A over 5 steps. Fig. 3 shows the matrix sharpening step by step from no visible structure into A ’s concentration around state 3; Fig. 4 consolidates this quantitatively, with $H(M)$ descending monotonically from $\log_2 9 \approx 3.170$ bits – the

² FluidR3 SoundFont, grand piano preset.

³ <https://github.com/MicaAnt/MatrixGen/tree/main/MatrixGen/compositions/audio>

⁴ $H(M) = \frac{1}{N} \sum_{i=0}^{N-1} \left(- \sum_{j=0}^{N-1} M_{i,j} \log_2 M_{i,j} \right)$, with the convention $0 \log_2 0 = 0$. $H(M)=0$ for a fully deterministic matrix (every row one-hot); $H(M)=\log_2 N$ for the fully uniform matrix, the maximum possible. Because `MatRotate` only permutes probabilities within each row, it never changes $H(M)$ – two matrices related by rotation always have equal entropy.

⁵ *Gestos*’ original chain was controlled with `ml.markov` and the `bach` library, in Max/MSP.

⁶ `PlayChain` draws a sequence of states from a matrix, playing each state’s material, and returns the ending state – so calls stay continuous even as the matrix changes mid-piece; code source in footnote 1.

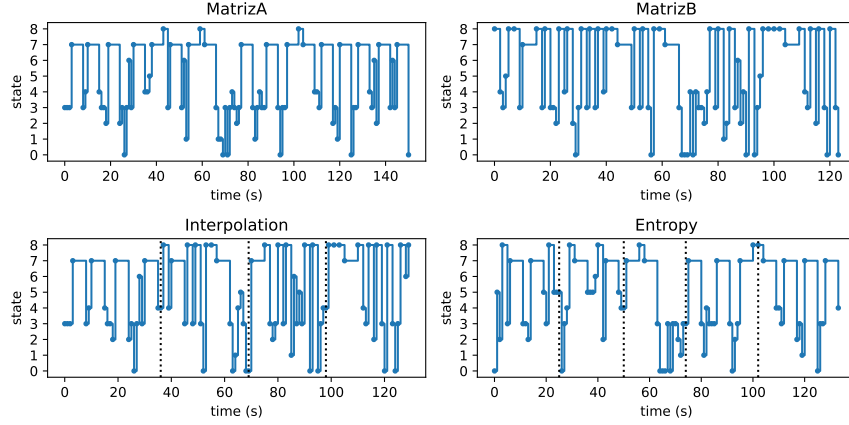


Fig. 2. State-over-time timeline for the four examples: A alone, B alone, the $A \rightarrow B$ interpolation, and the uniform $\rightarrow A$ entropy relaxation. Dotted vertical lines mark where the transition matrix changes mid-performance (segment boundaries); A and B alone have none, being single-matrix examples.

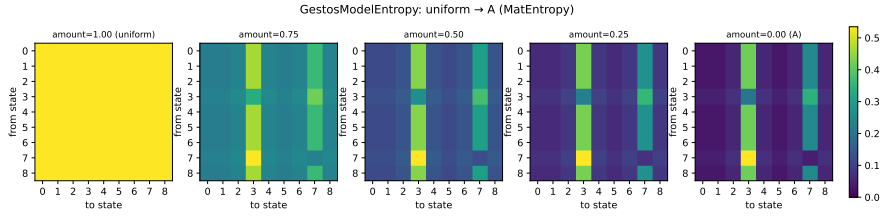


Fig. 3. Transition matrix at each of the 5 entropy-relaxation steps ($\alpha = 1.00, 0.75, 0.50, 0.25, 0.00$), from the uniform matrix (left) to A (right). Brighter (yellow) cells indicate higher transition probability.

theoretical maximum for $N=9$, reached exactly at the uniform matrix – down to A 's ≈ 2.488 bits, giving the composer a measured axis for how loosened or directed the texture is at each step.

4 Discussion and Conclusion

As a composer working with technology, I prefer to build my own algorithms, not because they are better, but because I compose more freely with tools I made myself. The four operations crystallise procedures drawn from my own trajectory: my use of Markov chains was born from the textural demands of earlier pieces (*Rarefações* and *Gestos*), and is distilled here into reusable operations that recur across pieces rather than being re-derived inline for each one. Because they act on the chain that governs the succession of events, the same tools couple to symbolic scores and to audio or synthesis processes alike, as in my generative work [7]. What draws me to this formalisation is that it bears

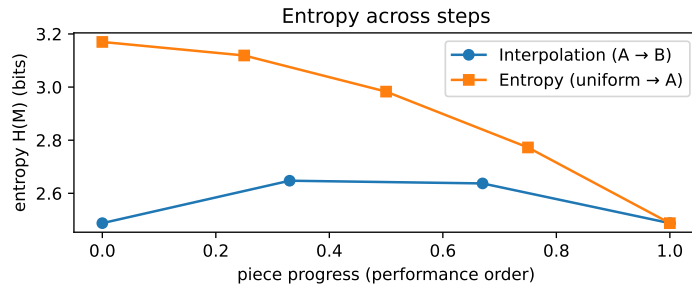


Fig. 4. Average row-wise Shannon entropy $H(M)$ at each step, plotted in performance order (0 = first segment played, 1 = last). Interpolation moves $A \rightarrow B$; Entropy moves uniform $\rightarrow A$.

on perception: fixing both the material and the design of the chain shapes what is actually heard. Tracking $H(M)$ across a piece adds a measurable handle, turning determinacy and indeterminacy into an axis I can read as well as hear. I assessed these behaviours by listening to the rendered examples myself, checking that the audible sense of direction loosened and tightened as $H(M)$ predicted. The scope is deliberately narrow: each example exercises a single operation on one shared chain, entropy is only one lens, and the privilege-shift and exponential variants remain to be explored in composition.

This implementation is an intermediate step: it looks back at the works I have composed and prepares a high-level control for new ones, keeping composition, not algorithmic validation, as the lens. I next intend to use these operations within a new piece, and to add a further level of Markov chains that governs the operations themselves. In doing so I hope to contribute a composer's perspective to computer-assisted composition, algorithmic composition, and sound design.

References

1. Ames, C.: The Markov Process as a Compositional Model: A Survey and Tutorial. *Leonardo* **22**(2), 175–187 (1989), <https://muse.jhu.edu/pub/6/article/602251>, publisher: The MIT Press
2. Antunes, M., Misina, G., Manzolli, J.: Criação e Performance na Elaboração Colaborativa a Distância de Rarefações (2020). In: dos Santos, R.d.C.D., Carneiro, M., Rossetti, D. (eds.) *Pesquisa em arte, mídias e tecnologia: textos selecionados*, pp. 61–72. Stricto Sensus Editora, Rio Branco, Acre (2021)
3. Ghisi, D., Agostini, A.: Extending bach: A Family of Libraries for Real-time Computer-assisted Composition in Max. *Journal of New Music Research* **46**(1), 34–53 (Jan 2017). <https://doi.org/10.1080/09298215.2016.1236823>, <https://www.tandfonline.com/doi/full/10.1080/09298215.2016.1236823>
4. Highams, L., Olszewska, J.I.: Markov-Chain-Based Musical Creative Intelligent Agent Passing Successfully the Turing Test. In: 2023 IEEE 27th International Conference on Intelligent Engineering Systems (INES). pp. 000215–000220 (Jul 2023). <https://doi.org/10.1109/INES59282.2023.10297645>, <https://ieeexplore.ieee.org/abstract/document/10297645>, ISSN: 1543-9259
5. Hsu, C., Greer, R.: Comparative Assessment of Markov Models and Recurrent Neural Networks for Jazz Music Generation (Sep 2023). <https://doi.org/10.48550/arXiv.2309.08027>, <http://arxiv.org/abs/2309.08027>, arXiv:2309.08027 [cs.SD]
6. Lazzarini, V., Yi, S., Heintz, J., Brandtsegg, Ø., McCurdy, I.: Csound: a sound and music computing system. Springer (2016), <https://books.google.com/books?hl=pt-BR&lr=&id=GcmADQAAQBAJ&oi=fnd&pg=PR7&dq=Csound:+A+Sound+and+Music+Computing+System&ots=2jQBVfycC9&sig=cn5jdEQs4MJ2CJjODNX-WjoQ3M4>
7. Manzolli, J., Antunes, M.: Re (M) ind: Composition and Generative Sound Design of a Multimodal Opera. In: Proceedings of the XXVII Generative Art conference. Domus Argenia Publisher (2024), <https://hal.science/hal-04894588/>
8. Privato, N., Rampado, O., Novello, A.: A Creative Tool for the Musician Combining LSTM and Markov Chains in Max/MSP. In: Martins, T., Rodríguez-Fernández, N., Rebelo, S.M. (eds.) *Artificial Intelligence in Music, Sound, Art and Design*. pp. 228–242. Springer International Publishing, Cham (2022). https://doi.org/10.1007/978-3-031-03789-4_15
9. Sicuan, S.R.: Markov-Chain Melody: A Mathematical Approach to AI-Assisted Songwriting. In: *Artificial Intelligence in Music Production: Innovations, Practices, and Industry Implications*, pp. 117–142. IGI Global Scientific Publishing (2026). <https://doi.org/10.4018/979-8-3373-6279-3.ch005>, <https://www.igi-global.com/chapter/markov-chain-melody/www.igi-global.com/chapter/markov-chain-melody/407475>
10. Antunes da Silva, M.: Explorando Texturas Sonoras: Análise e Composição com Suporte de Descritores de Áudio e Modelos Computacionais. Tese de Doutorado, Universidade de Campinas, Campinas, Brasil (2024)
11. Verbeurgt, K., Dinolfo, M., Fayer, M.: Extracting Patterns in Music for Composition via Markov Chains. In: Orchard, B., Yang, C., Ali, M. (eds.) *Innovations in Applied Artificial Intelligence*. pp. 1123–1132. Springer, Berlin, Heidelberg (2004). https://doi.org/10.1007/978-3-540-24677-0_115
12. Wu, Y., Liu, C.: Design of Piano Automatic Accompaniment System Based on Markov Model. In: Tshirintzis, G.A., Favorskaya, M.N., Kountchev, R., Patnaik, S. (eds.) *Advances in Computational Vision and Robotics*. pp. 3–10. Springer Nature Switzerland, Cham (2023). https://doi.org/10.1007/978-3-031-38651-0_1
13. Xenakis, I.: Formalized music: thought and mathematics in composition. No. 6, Pendragon Press (1992)